קטע קוד A

```
class A
{
	int x;
public:
	A(int x=5) : x(x) { cout << "A: " << x << "\n"; }
	A(const A& other) : x(other.x) { cout << "A: (copy)\n"; }
	~A() { cout << "~A: " << x << "\n"; }

	virtual void toOs(ostream& os) const {}

	friend ostream& operator<<(ostream& os, const A& a)
	{
		os << a.x;
		a.toOs(os);
		return os;
	}
};

template<class T>
class B : public A
{
	T val;
public:
	B(const T& t, int x = 3) : A(x), val(t) {cout << "B: " << val << "\n"; }
	B(const B& other) : val(other.val) { cout << "B: (copy)\n"; }
	~B() { cout << "~B: " << val << "\n"; }

	virtual void toOs(ostream& os) const override
	{
		os << " " << val << endl;
	}
};
```

להלן מספר main אפשריים:

A1 main

```
int main()
{
   B<const char*> b1("hi", 8);
   B<const char*> b2(b1);
}
```

A2 main

```
int main()
{
   B<A> b(A(7));
}
```

A3 main

```
int main()
{
      B<A*> b1(new A(8), 9);
      cout << "------------------\n";
      A a(8);

      B<A*> b2(&a, 9);
      cout << "------------------\n";

      cout << b1 << endl;
      cout << b2 << endl;
      cout << "------------------\n";
}
```

A45 main

```
int main()
{
   B<const char*> b1("hi", 8);
   cout << "------------------\n";
   B<B<const char*>> b2(b1);
   cout << "------------------\n";
}
```

A6 main

```
int main()
{
   A* a1 = new B<int>(4, 7);
   delete a1;
}
```

קטע קוד B

```
class A
{
    static int counter;
    int x;
public:
    A(int x=++counter) : x(x) { cout << "c'tor x=" << x << endl; }
    A(const A& other) : x(other.x) { cout << "copy x=" << x << endl; }
    ~A() { cout << "d'tor x=" << x << endl; --counter;}

    static int getCounter() { return counter; }

    friend ostream& operator<<(ostream& os, const A& a)
    {
          os << a.x << " " << a.counter << endl;
          return os;
    }
};

int A::counter = 0;

int main()
{
   {
      vector<A> arr;
      arr.reserve(10);  // sets the physical size to 10, logical size
                        // remains unchanged -- zero in this case

      A a1(1);
      cout << "*** " << A::getCounter() << endl;
      cout << "1----------\n";
      arr.push_back(A());

      cout << "2----------\n";
      cout << "*** " << A::getCounter() << endl;
      arr.push_back(a1);
      cout << "*** " << A::getCounter() << endl;
      cout << "3----------\n";
      arr.push_back(A());
      cout << "4----------\n";
      cout << "*** " << A::getCounter() << endl;
   }
   cout << "*** " << A::getCounter() << endl;
   cout << "5----------\n";

   system("pause");
}
```

שימו לב –

1. הפונקציה push_back יוצרת עותק של האובייקט אליה.
2. הניחו שהקוד קומפל ללא אופטימיזציות.

קטע קוד C – שאלות בונוס

```
class G
{
    const char* s;
public:
    G(const char* s) : s(s) { cout << "G: " << s << endl; }

    virtual void foo() const { cout << "G: foo\n"; }
    virtual void goo() const = 0;
};

class F1 : virtual public G
{
    int x;
public:
    F1(int x = 9, const char* s = "kuku") : G(s), x(x)
                                { cout << "F1: " << x << endl; }

    virtual void goo() const override { foo();  cout << "F1: goo\n"; }
};

class F2 : virtual public G
{
    int y;
public:
    F2(int y = 8, const char* s = "bla") : G(s), y(y)
                                 { cout << "F2: " << y << endl; }

    virtual void moo() const { foo();  cout << "F2: koo\n"; }
};

class S12 : public F1, public F2
{
    float f;
public:
    S12(float f, const char* s, int x, int y) : f(f), F2(y), F1(x), G(s)
                                 { cout << "S12: " << f << endl; }
    void koo() const { foo(); goo(); moo(); }
};
```

להלן מספר main אפשריים:

C1 main

```
int main()
{
	S12 s(6.7f, "hi", 2, 3);
}
```

C2 main

```
class S1 : public F1
{
public:
	S1(const char* s, int x, int y) : F1(y), G(s) { cout << "S1\n"; }

	void koo() const { foo(); goo(); }
};

int main()
{
	S1 s("hi", 1, 2);
	s.koo();
}
```

C3 main

```
class S2 : public F2
{
public:
	S2(const char* s, int x, int y) : F2(y), G(s) { cout << "S2\n"; }

	void koo() const { foo(); moo(); }
};

int main()
{
	S2 s("hi", 1, 2);
	s.koo();
}
```

בהצלחה !